

Machine Construction: Evolutionary Search for Mathematical Objects and Algorithms

DSA5210 Session 10 —AI for Mathematics

11 June 2026

**A construction problem: 26 circles
in a unit square**

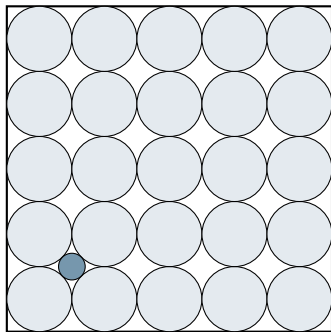
26 circles in a unit square

Problem. Place 26 circles inside the unit square:

- no two circles overlap (touching is allowed);
- no circle crosses the boundary;
- the radii need not be equal.

Maximize the sum of the 26 radii.

Best published placement (2025): radius sum 2.635, found by a computer search.



A placement made by hand — radius sum 2.541.

The by-hand placement: radius sum 2.541

- 25 circles of radius 0.1, centers at $(0.1 + 0.2i, 0.1 + 0.2j)$ for $i, j \in \{0, \dots, 4\}$:
 - neighbouring centers are 0.2 apart $= 0.1 + 0.1$ — the circles touch but do not overlap;
 - the outer circles touch the sides of the square.

Radius sum so far: $25 \times 0.1 = 2.5$.

- One more circle in a gap, centered at $(0.2, 0.2)$:
 - the four nearest centers are at distance $\sqrt{0.1^2 + 0.1^2} \approx 0.1414$;
 - $0.1 + 0.041 = 0.141 < 0.1414$ — a circle of radius 0.041 fits.
- Total radius sum: $2.5 + 0.041 = 2.541$.

Distance to the record: $2.635 - 2.541 = 0.094$.

Checking a placement is exact arithmetic

A candidate is a list of 26 triples (x_i, y_i, r_i) — 78 numbers. Checking it is a finite list of inequalities:

- inside the square: $r_i \leq x_i \leq 1 - r_i$ and $r_i \leq y_i \leq 1 - r_i$ for each i ;
- no overlap: $(x_i - x_j)^2 + (y_i - y_j)^2 \geq (r_i + r_j)^2$ for each of the $\binom{26}{2} = 325$ pairs.

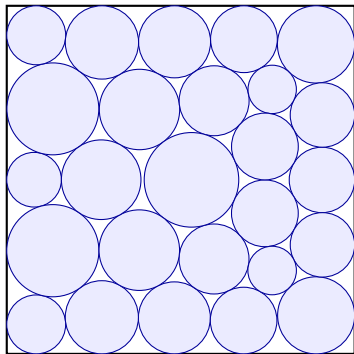
The check does not depend on who or what proposed the placement.

The two lower bounds: 2.541 and 2.635

- The by-hand placement proves: the maximum radius sum is at least 2.541.
- The 2025 placement (right) proves: it is at least 2.635.
- Whether a placement with a larger sum exists: open.

A record proves a bound; optimality stays open.

The placement drawn here is the published 2025 record itself (26 circles, radius sum 2.6358); checking it — every circle inside, every pair disjoint — is the finite arithmetic described on the previous frames, and was re-done independently for this lecture.



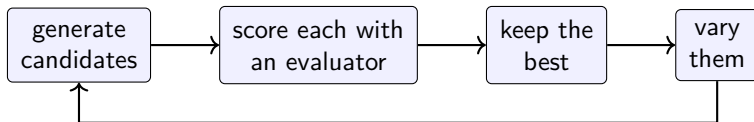
Construction problems

The circle problem, in general terms — a **construction problem** has three ingredients:

- a **search space**: a finite (or finitely described) set of candidate objects — here, placements of 26 circles;
- a **score function**: a computable number attached to each candidate — here, the sum of the radii;
- a **known record**: the best score any published construction achieves — here, 2.635.

The goal: an explicit candidate that scores past the record. Nothing needs to be proved about the optimum. In mathematical terms this is **extremal combinatorics**: maximize (or minimize) a score over a combinatorial class — here, sum of radii over packings.

Evolutionary search: generate → score → select → vary



- An **evolutionary search** keeps a pool of candidates and repeats the four steps.
- The **evaluator** is the program computing the score function — for the circles: check the inequalities, return the radius sum.
- The loop calls the evaluator thousands to millions of times per run, so scoring must be automatic and cheap — and a wrong evaluator makes the search optimize the wrong thing.

**First systems: Wagner,
AlphaTensor, AlphaDev
(2021–2023)**

Conjectured inequalities in spectral graph theory

A. Z. Wagner, *Constructions in combinatorics via neural networks*, [arXiv:2104.14516](https://arxiv.org/abs/2104.14516) (April 2021).

- Spectral graph theory studies a graph through eigenvalues of matrices attached to it — first of all the **adjacency matrix**: the $n \times n$ matrix with a 1 in entry (i, j) when vertices i and j are joined by an edge, 0 otherwise.
- Conjectures are produced here in bulk by computer search: **AutoGraphiX** (Caporossi–Hansen) hunts for inequalities among graph invariants that hold on every example it can generate; the Aouchiche–Hansen survey (*Linear Algebra Appl.* 432, 2010) collects conjectures produced this way.
- A conjectured inequality is a universal claim; one explicit violating graph settles it.



Adam Zsolt Wagner —
research scientist, Google
DeepMind; previously assistant
professor of mathematics,
Worcester Polytechnic
Institute. Photo: personal
homepage.

The target conjecture: $\lambda_1 + \mu \geq \sqrt{n-1} + 1$

Conjecture 2.1 of the survey (originally produced by AutoGraphiX): for every connected graph on $n \geq 3$ vertices,

$$\lambda_1 + \mu \geq \sqrt{n-1} + 1,$$

where

- λ_1 is the largest eigenvalue of the adjacency matrix;
- μ is the **matching number**: the maximum number of pairwise disjoint edges.

As a construction problem:

- search space: graphs on n vertices;
- score of a graph: $\sqrt{n-1} + 1 - \lambda_1 - \mu$ — the amount of violation;
- any graph with positive score is a counterexample.

Encoding and objective

Input a conjectured inequality, and an encoding of the candidate as a word — a graph on n vertices is a 0–1 string of length $\binom{n}{2}$, one bit per potential edge.

Output an explicit graph violating the inequality, if the search finds one.

Objective maximize the violation score, treated as a black box.

The graph is built bit by bit — edge slot by edge slot, each bit drawn from a probability that depends on the edges already placed.

What the network represents: a distribution over graphs

There are $2^{\binom{n}{2}}$ graphs on n vertices — too many to score one by one. The search needs to spend its trials where high-scoring graphs are likely, and to move that focus as it learns.

- The device for this is a **probability distribution over graphs we can sample from and adjust**. The network *is* that distribution.
- A graph is built one edge-bit at a time, so the distribution is fixed by one number per bit: the probability this edge is 1 *given the edges already decided*. That single conditional probability is the network's output.
- The network answers one question, once per edge slot: given the partial graph, how likely should this next edge be? Nothing about the conjecture is built in — it starts uniform (≈ 0.5) and only the score reshapes it.

What the output p is used for: it is the bias of a coin flip for this one edge; $\binom{n}{2}$ flips produce one graph; 1000 graphs make a batch to score. The *graphs* are the deliverable — the network is only the mechanism pulled toward where the best graphs sit, then discarded.

The proposal network: a multilayer perceptron

Fix an order on the $\binom{n}{2}$ edge slots and decide them one bit at a time. At step $t \in \{0, 1, \dots, \binom{n}{2} - 1\}$ the network sees two vectors over the same $\binom{n}{2}$ slots.

- **A — the partial graph.** $A \in \{0, 1\}^{\binom{n}{2}}$. For $i < t$, $A[i]$ is the bit already chosen for edge slot i . For $i \geq t$, $A[i] = 0$ (placeholder; slot not yet decided).
- **B — the position pointer.** $B \in \{0, 1\}^{\binom{n}{2}}$ with $\sum_i B[i] = 1$ (one-hot). The unique 1 sits at position t : “decide edge slot t now”. Linking constraint: $A[i] = 0$ whenever $B[j] = 1$ with $j \leq i$ — the network sees no edges past the pointer.
- **Concatenate (A, B) :** $2\binom{n}{2}$ inputs (342 for $n = 19$).
- **Hidden:** dense 128, dense 64, dense 4; ReLU $x \mapsto \max(0, x)$ on each layer.
- **Output:** one sigmoid neuron $\rightarrow p \in (0, 1)$. Sample $b \sim \text{Bernoulli}(p)$, set $A[t] \leftarrow b$, move B 's 1 from position t to position $t+1$, query again. After $\binom{n}{2}$ queries one graph is finished.
- **Training:** binary cross-entropy on the bit decisions taken in the best-scoring sampled graphs.

The cross-entropy method loop

A sample–select–imitate loop around the network:

1. The network reads the partial word and the current position, and outputs the probability that the next bit is 1.
2. Sample a batch of complete constructions (1000 per iteration in the released code) bit by bit from these probabilities.
3. Score every construction; keep the **elite** — the top few percent (93rd percentile in the released code).
4. Retrain the network to imitate the elite's decisions; carry the very best constructions into the next batch.
5. Repeat — the proposal distribution concentrates on high-scoring structures.

A 19-vertex counterexample

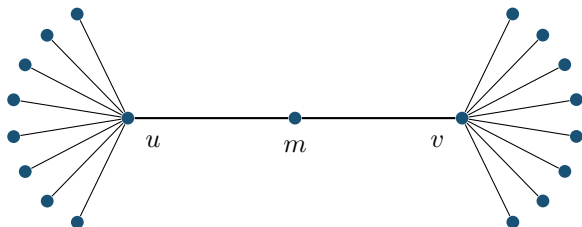
On Conjecture 2.1 ($\lambda_1 + \mu \geq \sqrt{n-1} + 1$):

- The agent found a 19-vertex counterexample — a tree Wagner calls a *balanced double star* — with $\lambda_1 = \sqrt{10}$, $\mu = 2$:

$$\lambda_1 + \mu \approx 5.16 < 5.24 \approx \sqrt{18} + 1.$$

- The conjecture holds for all $n \leq 18$, so no smaller counterexample exists.
- History, stated in Wagner's paper: Stevanović had already disproved the conjecture in 2010, with a 600-vertex counterexample.

The counterexample: two stars joined through a middle vertex



- A middle vertex m joined to the centers u, v of two stars with 8 leaves each: 19 vertices.
- $\mu = 2$: every edge contains u or v , so a matching has at most two edges; one leaf edge at u plus one at v are disjoint.
- $\lambda_1 = \sqrt{10}$: in the top eigenvector the centers carry equal value x ; each leaf carries x/λ , the middle vertex $2x/\lambda$; the center equation $\lambda x = 8(x/\lambda) + 2x/\lambda$ forces $\lambda^2 = 10$.

Seven problems; the 203-vertex counterexample

- The paper refutes or answers seven problems: five by this loop, two by linear programming.
- Among them: a second conjecture from the same survey, about the eigenvalues of the graph's distance matrix, refuted at 203 vertices; a 1989 conjecture of Collins on characteristic-polynomial coefficients of trees.
- Wagner: “the only thing we need to change in the code is the function that calculates the score of a given construction.”

The 203-vertex case:

- The run at $n = 30$ found *no* counterexample.
- Its near-optimal structures showed a pattern — a path with many pendant vertices (vertices of degree 1).
- Wagner scaled the pattern up by hand to 203 vertices, where it violates the conjecture.

Next problem: fast matrix multiplication

Wagner's loop searched for a *graph*. AlphaTensor (DeepMind, 2022) searches for an *algebraic algorithm*.

- **Concrete instance:** 2×2 . Compute $C = AB$ for 2×2 matrices over a field $\mathbb{F}$. The textbook formula uses 8 multiplications $a_{ij}b_{k\ell}$. Can fewer than 8 scalar multiplications do it?
- **Search space.** A *bilinear scheme with r multiplications*: three coefficient tables $\alpha_{ij}^{(k)}, \beta_{ij}^{(k)}, \gamma_{ij}^{(k)} \in \mathbb{F}$ ($k = 1, \dots, r; i, j \in \{1, 2\}$) defining

$$m_k = \left(\sum_{i,j} \alpha_{ij}^{(k)} a_{ij} \right) \left(\sum_{i,j} \beta_{ij}^{(k)} b_{ij} \right), \quad c_{ij} = \sum_{k=1}^r \gamma_{ij}^{(k)} m_k.$$

Each m_k is one multiplication in $\mathbb{F}$. The α, β, γ are the unknowns to search for; the scheme is valid iff $c_{ij} = (AB)_{ij}$ as polynomials in a_{ij}, b_{ij} .

- **Score: r , the multiplication count.** Applied recursively, the 2×2 scheme multiplies $N \times N$ matrices in $\Theta(N^{\log_2 r})$ time — r becomes the exponent on N ; block-additions add only a constant factor. Lower r = lower exponent.
- **Known record.** Strassen 1969, $r = 7$ for 2×2 (next frame): $\Theta(N^{2.807})$ vs textbook $\Theta(N^3)$.

Strassen 1969: 2×2 matrices in 7 multiplications

Instead of the 8 products of the textbook formula, compute

$$\begin{aligned}m_1 &= (a_{11} + a_{22})(b_{11} + b_{22}) & m_5 &= (a_{11} + a_{12})b_{22} \\m_2 &= (a_{21} + a_{22})b_{11} & m_6 &= (a_{21} - a_{11})(b_{11} + b_{12}) \\m_3 &= a_{11}(b_{12} - b_{22}) & m_7 &= (a_{12} - a_{22})(b_{21} + b_{22}) \\m_4 &= a_{22}(b_{21} - b_{11})\end{aligned}$$

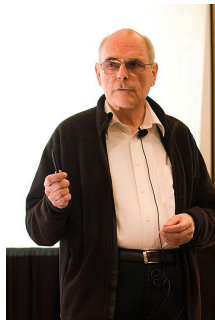
and assemble the product from them:

$$\begin{aligned}c_{11} &= m_1 + m_4 - m_5 + m_7, & c_{12} &= m_3 + m_5, \\c_{21} &= m_2 + m_4, & c_{22} &= m_1 - m_2 + m_3 + m_6.\end{aligned}$$

7 multiplications, 18 additions/subtractions, valid over any field.

Recursion on blocks: $O(n^{\log_2 7})$

- The entries a_{ij}, b_{ij} may themselves be matrix blocks — Strassen's identities use no commutativity.
- Applied recursively to 2×2 block matrices: 7 multiplications per level, so running time $O(n^{\log_2 7}) = O(n^{2.807})$ instead of $O(n^3)$.
- Additions are cheap ($O(n^2)$ per level); recursion amplifies the multiplication count, so the number of multiplications is the quantity to minimize.
- A 4×4 matrix, treated as a 2×2 matrix of 2×2 blocks: $7 \times 7 = 49$ multiplications.



Volker Strassen (b. 1936) — German mathematician; the 1969 seven-multiplication algorithm started fast matrix multiplication.
Photo: Wikimedia Commons.

The matrix-multiplication tensor $\mathcal{T}_n$

$C = AB$ is bilinear in (A, B) . Any bilinear map is described by a 3D table of structure constants; $\mathcal{T}_n$ is that table.

- **Flatten.** Read $A, B, C \in \mathbb{F}^{n \times n}$ row-by-row as $a, b, c \in \mathbb{F}^{n^2}$. For $n = 2$:
 $(a_1, a_2, a_3, a_4) := (a_{11}, a_{12}, a_{21}, a_{22})$.
- **Defining equations ($n = 2$).** From the textbook expansion of $C = AB$:

$$c_1 = a_1b_1 + a_2b_3, \quad c_2 = a_1b_2 + a_2b_4, \quad c_3 = a_3b_1 + a_4b_3, \quad c_4 = a_3b_2 + a_4b_4.$$

- **Tensor.** $\mathcal{T}_2 \in \{0, 1\}^{4 \times 4 \times 4}$ with $(\mathcal{T}_2)_{ijk} = \text{coefficient of } a_i b_j \text{ in } c_k$:

$$c_k = \sum_{i,j=1}^4 (\mathcal{T}_2)_{ijk} a_i b_j.$$

Eight ones: $(1, 1, 1), (2, 3, 1), (1, 2, 2), (2, 4, 2), (3, 1, 3), (4, 3, 3), (3, 2, 4), (4, 4, 4)$;
other 56 entries are 0.

- **General n .** $\mathcal{T}_n \in \{0, 1\}^{n^2 \times n^2 \times n^2}$, n^3 ones.

Rank- R decomposition = algorithm with R multiplications

$u, v, w \in \mathbb{F}^{n^2}$ are vectors of coefficients. Their **outer product** is the rank-one tensor with entries $(u \otimes v \otimes w)_{ijk} = u_i v_j w_k$. A **rank- R decomposition** of $\mathcal{T}_n$ is

$$\mathcal{T}_n = \sum_{r=1}^R u^{(r)} \otimes v^{(r)} \otimes w^{(r)} \iff (\mathcal{T}_n)_{ijk} = \sum_{r=1}^R u_i^{(r)} v_j^{(r)} w_k^{(r)}.$$

Read columnwise it is an R -multiplication algorithm for $C = AB$:

$$m_r = \left(\sum_i u_i^{(r)} a_i \right) \left(\sum_j v_j^{(r)} b_j \right), \quad c_k = \sum_r w_k^{(r)} m_r.$$

- $u^{(r)}, v^{(r)}, w^{(r)}$ are exactly $\alpha^{(r)}, \beta^{(r)}, \gamma^{(r)}$ from the bilinear-scheme frame, with the matrix index (i, j) flattened to a single index in $\{1, \dots, n^2\}$.
- Strassen = rank-7 decomposition of $\mathcal{T}_2$: each r gives one m_r , the seven columns together reproduce all 64 entries of $\mathcal{T}_2$.
- Rank of $\mathcal{T}_n :=$ the minimal such R . Finding low-rank decompositions of a given tensor is NP-hard in general (Håstad 1990).

Known ranks before 2022; the exponent ω

- The rank of $\mathcal{T}_3$ is unknown: between 19 (Bläser's lower bound) and 23 (Laderman's 1976 scheme).
- Before 2022, nothing below 49 was known for 4×4 over any field.
- Separate research line: the asymptotic exponent ω — the smallest τ such that two $n \times n$ matrices can be multiplied in time $O(n^{\tau+\varepsilon})$ for every $\varepsilon > 0$ — currently $\omega < 2.371339$ (Alman–Duan–Vassilevska Williams–Xu–Xu–Zhou, SODA 2025). Those algorithms hide constants so large they never beat practical algorithms at real sizes.

TensorGame: decomposition as a single-player game

Fawzi et al., *Discovering faster matrix multiplication algorithms with reinforcement learning*, Nature 610 (5 Oct 2022) 47–53 — DeepMind's system **AlphaTensor**.

Target: exact, usable decompositions at small fixed sizes — not the exponent ω .

Input the tensor $\mathcal{T}_{n,m,p}$ and a finite coefficient set F — e.g.
 $F = \{-2, -1, 0, 1, 2\}$ for standard arithmetic, or $F = \{0, 1\}$ with reduction mod 2 for arithmetic in $\mathbb{Z}_2$.

Output triples $(u^{(r)}, v^{(r)}, w^{(r)})$ forming a verified rank- R decomposition — a correct algorithm with R multiplications.

Objective minimize R .

- **State:** the residual tensor, starting at $S_0 = \mathcal{T}_n$.
- **Move:** pick one triple (u, v, w) over F ; update $S_t = S_{t-1} - u \otimes v \otimes w$.
- **Reward:** -1 per move; if the move limit is hit, an extra penalty bounding the rank of the remainder.
- **End:** $S_t = 0$ — the moves played are a valid decomposition.

The agent: AlphaZero adapted to tensors

AlphaZero: Monte Carlo tree search (MCTS) guided by a network that proposes moves (policy) and values states (value), trained by self-play (reinforcement learning: trial and error driven by a numeric reward).

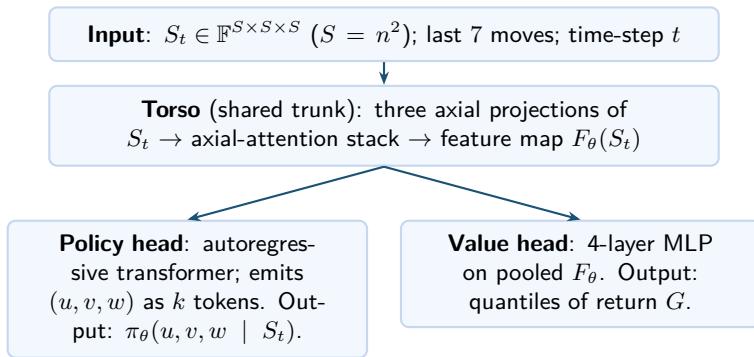
- Per-move action space above 10^{12} (the number of triples (u, v, w) over the coefficient set F ; board games: hundreds of legal moves) — actions are *sampled* from the policy network, never enumerated.
- **Synthetic demonstrations:** sum R random triples — a tensor with a known decomposition; pre-train on 5 million such pairs.
- **Change of basis:** re-express A, B, C in new coordinates; $\mathcal{T}_n$ changes but its rank does not, so any basis's decomposition converts back. Each game uses one of $\sim 100,000$ random bases.



Alhussein Fawzi — first author of AlphaTensor; led the work as a research scientist at Google DeepMind; PhD EPFL 2016. Photo: fawzi.ai.

AlphaTensor's network: shared trunk + two heads

One network $f_{\theta}(S_t) = (\pi_{\theta}, V_{\theta})$, produced by a shared *trunk* (Torso) and two task *heads* (Policy, Value).



Why this shape: at each MCTS visit the agent needs a move suggestion *and* a value of the state; sharing the trunk computes features once, two heads read them. **Axial attention:** on an $S \times S$ grid attend along one axis at a time — $O(S^3)$ per layer instead of $O(S^4)$.

AlphaTensor's network: what's optimized, what's trained

- **End goal (optimization target).** For a fixed $\mathcal{T}_n$, minimize R — the number of TensorGame moves to drive S_t to $\mathbf{0}$. Reward -1 per move, total return $G = -R$. The decomposition itself is produced *at inference*: MCTS guided by the trained network plays one game on $\mathcal{T}_n$; the recorded triples $(u^{(r)}, v^{(r)}, w^{(r)})$ are the decomposition.
- **Training targets (what the heads chase).**
 - Policy head $\pi_\theta(\cdot|S)$ chases the **MCTS improved policy** $\pi^*(\cdot|S)$ — the visit-count distribution over moves after MCTS searches from S . Loss: cross-entropy $L_\pi = -\sum_a \pi^*(a|S) \log \pi_\theta(a|S)$.
 - Value head $V_\theta(S)$ chases the **observed return** G from S . AlphaTensor models the full distribution of G ; L_V = quantile-regression loss.

Joint loss $L_\theta = L_\pi + L_V$; minimized by SGD on θ .

- **Training data (two streams, mixed in each batch).**
 - *Self-play*: the current network plays games against itself via MCTS; stored as (S, π^*, G) triples.
 - *Synthetic demos*: sum R random rank-one triples $\rightarrow$ a tensor with a *known* R -rank decomposition; the R summands are supervised (S, action) examples.

AlphaTensor: results

format	arithmetic	previous best	AlphaTensor
4×4	$\mathbb{Z}_2 \pmod{2}$	49	47
4×4	standard	49	49 (not improved)
4×5 by 5×5	standard	80	76
5×5	$\mathbb{Z}_2$	98	96

- The 47 is the first count below the 49 of Strassen applied twice, since 1969; the naive count for 4×5 by 5×5 is 100.
- Matched or improved the best known rank for over 70 tensors ($n, m, p \leq 12$).
- The rank-47 scheme uses mod-2 arithmetic, so it works only in characteristic 2; the 4×4 record over $\mathbb{R}, \mathbb{C}$ stayed at 49. Recursively it gives $O(N^{\log_4 47}) = O(N^{2.778})$ — a runnable exponent, unlike the smaller asymptotic $\omega < 2.371$ (only for matrices too large to compute).
- At rank 49 (standard arithmetic) it found more than 14,000 inequivalent schemes.

Why a mod-2 scheme is not an algorithm for real matrices

A multiplication scheme is a set of *algebraic identities*: the products m_r and the assembly $c_k = \sum_r w_k^{(r)} m_r$ must expand to the right entries of AB . Whether an identity is *true* depends on the arithmetic the entries obey.

- Strassen uses only the coefficients $+1, -1$; the identities hold by ordinary algebra valid in **every** field — real, complex, rational, mod- p alike.
- A scheme found “mod 2” may use $1 + 1 = 0$ (equivalently $x + x = 0$): an identity leaning on that is true mod 2 but **false over the reals**. So AlphaTensor’s 47 is a correct algorithm only where $1 + 1 = 0$ holds.

The **characteristic** of a field is how many times you add 1 to reach 0:

- characteristic 2: $1 + 1 = 0$ — e.g. $\mathbb{F}_2 = \{0, 1\}$, the arithmetic of bits (coding theory, cryptography);
- characteristic 0: never returns to 0 — $\mathbb{Q}, \mathbb{R}, \mathbb{C}$, the matrices people actually compute with.

Over characteristic 0 the 4×4 record stayed at Strassen’s 49 — the barrier AlphaEvolve broke.

Kauers–Moosbauer: $96 \rightarrow 95$ in three days

- Kauers and Moosbauer ([arXiv:2210.04045](https://arxiv.org/abs/2210.04045), 8 October 2022 — three days after the Nature paper): starting from AlphaTensor's 5×5 mod-2 scheme, hand-guided transformations remove one more multiplication: 95.
- They systematized the idea as random walks on *flip graphs* — vertices are correct multiplication schemes, edges are local modifications (ISSAC 2023) — improving further formats over arbitrary fields.
- Jakob Moosbauer: doctoral work at the same institute (Johannes Kepler University Linz).



Manuel Kauers — professor of algebra, Johannes Kepler University Linz; head of its Institute for Algebra. Photo: JKU.

Hardware-tailored decompositions

- Change the terminal reward to the measured runtime of the found algorithm on a target device — multiplying 8192×8192 matrices as 4×4 blocks.
- Targets: an Nvidia V100 GPU and a Google TPU v2 (tensor processing unit, Google's machine-learning accelerator chip).
- Found schemes run 10–20% faster (DeepMind's reported range) than the standard library multiplication on large matrices.
- Schemes tuned for one device lose their advantage on the other.

What AlphaTensor released

github.com/google-deepmind/alphatensor — code Apache 2.0, other materials CC-BY 4.0. Four directories:

- `algorithms/` — the discovered factorizations as compressed array files (the `.npz` format of NumPy, Python's array library; one file for standard arithmetic, one for mod 2), plus a notebook that loads and verifies them.
- `benchmarking/` — the speed measurement of the discovered 4×4 -block algorithm on an Nvidia V100 GPU.
- `nonequivalence/` — the 14,236 rank-49 factorizations of $\mathcal{T}_4$, with invariants certifying pairwise nonequivalence.
- `recombination/` — code building larger formats from factorizations of smaller ones.

Not included: any training or search code. The paper's Methods give the withheld system's scale: training on 64 TPU v3 cores for 600,000 iterations, self-play on 1,600 TPU v4 actors (workers that play games to generate training data), about one week to converge.

AlphaDev: sorting routines as a game over assembly

Mankowitz et al., *Faster sorting algorithms discovered using deep reinforcement learning*, Nature 618 (June 2023) 257–263.

The object constructed is a program for sorting a small fixed number of values, built one x86 assembly instruction at a time (assembly: the processor's lowest-level instruction language).

- **State:** the program so far, plus the contents of registers (the processor's working storage) and memory after running it on test inputs.
- **Move:** append one instruction (mov = copy, cmp = compare, cmov = copy only if a condition holds — the branchless conditional that makes the discovered sorts fast).
- **Reward:** correctness on test sequences, plus a latency term (program length or measured latency).
- **Agent:** an AlphaZero extension, as in TensorGame.

AlphaDev: results and deployment

Program length in instructions, against the human benchmark:

routine	human	AlphaDev
sort3	18	17
sort4	28	28 (matched)
sort5	46	42
VarSort5 (variable length)	115	63

- The routines were translated back to C++ and merged into libc++ — the C++ standard library of the LLVM compiler infrastructure — in April 2022.
- First change to these small-sort subroutines in over a decade; the paper estimates the code runs trillions of times per day.



Daniel J. Mankowitz — first author of AlphaDev; staff research scientist, Google DeepMind. Photo: X profile.

Two limits shared by Wagner, AlphaTensor, AlphaDev

- **Move-level generation.** The generator emits one bit / one rank-one tensor / one instruction at a time, starting from zero knowledge.
 - Everything the search knows, it learned from the score.
 - No mathematical literature and no domain idiom enters the search.
- **One problem, one system.** AlphaTensor's network, game, and action space are purpose-built for tensors; AlphaDev's for x86 assembly.
 - A new problem means a new encoding, a new action space, often a new network.

FunSearch: a language model as the generator

Cap sets in $\mathbb{F}_3^n$

$\mathbb{F}_3^n$: the length- n vectors with entries 0, 1, 2, added coordinate-wise mod 3. A **cap set** in $\mathbb{F}_3^n$ is a set of vectors no three of which (distinct) sum to zero. Equivalent formulations:

- Since $2 \equiv -1 \pmod{3}$: $x + y + z = 0 \iff x + z = 2y$ — no 3-term arithmetic progression.
- Lines of the affine space $\text{AG}(n, 3)$ are exactly the triples $\{a, a + d, a + 2d\}$ — a cap set contains no full line.

The card game SET is the case $n = 4$:

- the 81 cards are the points of $\mathbb{F}_3^4$ (four attributes, three values each);
- a “SET” is a line;
- the largest collection of cards containing no SET has exactly 20 cards (Pellegrino, 1971).

Known maximum sizes c_n

c_n = maximum size of a cap set in dimension n ; exact values known only up to $n = 6$:

n	1	2	3	4	5	6	7	8
c_n	2	4	9	20	45	112	?	?
known range							236–288	≥ 496 (pre-2023)

- $c_6 = 112$: construction from Hill's 1973 cap; maximality proved by Potechin (2008).
- $n = 7$: best known construction 236 points.
- Brute force in dimension 8 is out of reach: $3^8 = 6561$ vectors, $\binom{6561}{512} \approx 10^{779}$ candidate sets of size 512.

The cap set capacity C

Define $C = \sup_n c_n^{1/n}$, the growth rate of maximum cap sets.

From above: $C \leq 2.756$ — Croot–Lev–Pach, then Ellenberg–Gijswijt (*Annals* 185, 2017), polynomial method.

From below — one explicit cap set $S \subset \mathbb{F}_3^n$ per row, giving $C \geq |S|^{1/n}$. A **construction problem on cap sets** (score = $|S|$):

bound	construction	year
$C \geq 2.2101$	Calderbank–Fishburn	1994
$C \geq 2.2173$	Edel	2004
$C \geq 2.2180$	Tyrrell (SAT solvers)	2022



Terence Tao — UCLA, Fields Medal 2006. On the cap set problem: “perhaps my favourite open question.” Photo: Wikimedia Commons.

FunSearch: input, output, objective

Romera-Paredes et al., *Mathematical discoveries from program search with large language models*, Nature 625 (online 14 Dec 2023) 468–475. “FunSearch” = searching in *function* space.

FunSearch trains no model. The LLM is used *frozen* as a code sampler; all improvement accumulates in a population of Python programs evolved by selection on the evaluator’s score. This is the break from Wagner/AlphaTensor/AlphaDev, where the network is trained from scratch on the score.

Input (a) an **evaluator**: a trusted scoring program — run the candidate, check the cap property exactly, return the size; (b) a **skeleton**: a short greedy program with one inner heuristic function left trivial; (c) a pretrained code **LLM**, Codey (Google’s PaLM 2, code-specialized) — used frozen as a black-box sampler.

Output a short Python function; plugged into the skeleton, it produces the record object.

Objective maximize the evaluator’s score of the skeleton + function combination.

The cap-set skeleton: only priority evolves

```
def solve(n):  
    cap = []  
    for v in sorted(F3n(n), key=lambda v: -priority(v, n)):  
        if not creates_line(cap, v):  
            cap.append(v)  
    return cap                                # the candidate cap set in  $\mathbb{F}_3^n$   
  
def priority(v, n):                          # <- the ONLY part the LLM rewrites  
    return 0.0
```

- `solve(n)` returns an explicit cap set $S \subset \mathbb{F}_3^n$. The FunSearch **score** is its size: $|S| = \text{len}(\text{solve}(n))$.
- Enumeration, the line-check `creates_line`, and the greedy loop are human-written — correct by construction.
- The LLM only proposes new `priority(v, n)` bodies. Search space = functions $\mathbb{F}_3^n \rightarrow \mathbb{R}$; a better ordering yields a larger greedy cap set.

The evolutionary loop

1. **Sample.** Show the LLM $k = 2$ stored programs sorted by score (priority_v0, priority_v1); ask it to write priority_v2 (“best-shot prompting”).
2. **Evaluate.** Run each generated program in a sandbox with time and memory limits; discard crashes, timeouts, and low scores.
3. **Store.** Keep scored programs in a database of 10 **islands** — independent subpopulations.
 - Every 4 hours, wipe the worst half of the islands; reseed from the best survivors.
 - Within an island, cluster programs by behavior signature (the tuple of scores on the test inputs); prefer shorter programs among equals.
4. Repeat, asynchronously.

Scale and cost

- About 15 LLM samplers and ~ 150 CPU evaluators in parallel.
- On the order of 10^6 generated programs per experiment, over a few days.
- The LLM is never trained — all improvement accumulates in the population of programs.
- Reproduction estimate from the paper, with the open code model StarCoder-15B (15 billion parameters): about two days on 15 GPUs plus 5 CPU servers, roughly \$800–1,400 of cloud compute.

Dimension 8: a 512-point cap set

- FunSearch found a cap set of size **512** in $\mathbb{F}_3^8$.
- Previous record: 496, from a 2001–2004 line of constructions combining lower-dimensional caps (Edel–Bierbrauer, Edel).
- FunSearch matched the known records for all $n \leq 7$.
- The 512 was rare even for FunSearch: 4 out of 140 runs reached it.



Bernardino Romera-Paredes —
first author of FunSearch;
research scientist, Google
DeepMind; PhD UCL, postdoc
Oxford. Photo: X profile.

Admissible sets: cap-set templates

Shift of target. A cap set in dim n gives one row of the lower-bound table. An *admissible set* $\mathcal{A} \subset \{0, 1, 2\}^n$ of weight- w vectors (each with exactly w nonzero entries) gives, via a fixed substitution, a cap-set family in $\mathbb{F}_3^{nk}$ for every $k \geq 1$, hence one bound $C \geq |\mathcal{A}|^{1/n}$.

The substitution. Fix a small cap set $C_0 \subset \mathbb{F}_3^k$ with two disjoint “slot” subsets. Lift each template $v \in \mathcal{A}$ block by block: at block i place a zero block if $v_i = 0$, a slot-1 vector if $v_i = 1$, a slot-2 vector if $v_i = 2$. Varying the slot choices gives the cap-set family above (Calderbank–Fishburn 1994, Edel 2004).

Admissibility (what makes the lift a cap set). For any three distinct templates a, b, c , at some coordinate i the multiset $\{a_i, b_i, c_i\}$ is one of $\{0, 1, 2\}, \{0, 0, 1\}, \{0, 0, 2\}$ — i.e. zero + slot-1 + slot-2, or two zeros + one slot- j . At block i the three lifted blocks cannot form a 3-AP, so a, b, c are not a 3-AP globally.

Full-size: $|\mathcal{A}| = \binom{n}{w}$, written $\mathcal{I}(n, w)$. FunSearch evolves priority on $\{0, 1, 2\}^n$, same skeleton.

Three capacity improvements: $2.2184 \rightarrow 2.219486 \rightarrow 2.2202$

1. FunSearch found a full-size admissible set $\mathcal{I}(12, 7)$: $C \geq 2.2184$ — past Tyrrell's SAT-solver record.
2. The authors read the discovered program; it treated coordinates symmetrically in blocks. They defined symmetry-restricted admissible sets and re-ran the search in the smaller space: $\mathcal{I}(15, 10)$, giving $C \geq 2.219486$.
3. The restricted search produced a partial admissible set in $\mathcal{A}(24, 17)$ with 237,984 vectors:

$$C \geq 2.2202.$$

The paper: “the largest improvement in 20 years to the asymptotic lower bound” (i.e. since Edel 2004).

Program output vs. object output

- **Compression.** The 237,984-vector admissible set is generated by a program of a few lines. The paper: FunSearch “attempts to find solutions that have low Kolmogorov complexity” (the length of the shortest program producing the object); “because most problems we care about are structured (highly non-random), we believe that solutions are described more concisely with a computer program.”
- **Scalability.** The previous record ($\mathcal{I}(11, 7)$, Tyrrell 2022) came from SAT solvers, which enumerate the object and stop scaling; a program instantiates at sizes no explicit search reaches.
- **Readability.** The $\mathcal{I}(12, 7)$ program’s block symmetry was read by the authors and turned into a restricted search space — which then broke the record again.
- The paper on the LLM’s role: “a source of diverse (syntactically correct) programs with occasionally interesting ideas.”

Jordan Ellenberg on the discovered programs

“The solutions generated by FunSearch are far conceptually richer than a mere list of numbers. When I study them, I learn something.”

Jordan Ellenberg — University of Wisconsin–Madison; co-author of the Ellenberg–Gijswijt upper bound $C \leq 2.756$, and a co-author of the FunSearch paper.



Jordan Ellenberg. Photo: Wikimedia Commons.

Online bin packing with the same loop

Online bin packing: items of sizes in $(0, 1]$ arrive one at a time; each must be placed immediately into a bin of capacity 1; minimize the number of bins used.

- Classical baselines: *first fit* (leftmost bin that fits), *best fit* (fullest bin that fits).
- Skeleton: assign each arriving item to the bin maximizing an evolved `heuristic(item, bins)`.
- Benchmarks: OR-Library (Beasley's standard operations-research test instances) and instances with Weibull-distributed item sizes (a model of realistic scheduling workloads).

Bin packing: excess bins over the lower bound

Fraction of bins beyond the standard lower bound on the optimal offline packing — the best packing computable with all items known in advance (smaller is better):

	OR1	OR2	OR3	OR4	Weib. 5k	10k	100k
first fit	6.42%	6.45%	5.74%	5.23%	4.23%	4.20%	4.00%
best fit	5.81%	6.06%	5.37%	4.94%	3.98%	3.90%	3.79%
FunSearch	5.30%	4.19%	3.11%	2.47%	0.68%	0.32%	0.03%

- The discovered rule: place the item best-fit-style only when the fit is tight; otherwise prefer bins with more room left — avoiding small unfillable gaps.
- Trained on small instances; the same function generalizes to instances 100× larger.

FunSearch's scope conditions

Stated by the authors as scope conditions:

- an efficient evaluator;
- a graded score — partial credit, not yes/no;
- a skeleton isolating a small evolvable heuristic.

Problems lacking these are out of scope — the paper names theorem proving explicitly: the signal is binary, and there is no cheap evaluator short of a proof checker run on a complete proof.

What FunSearch released

`github.com/google-deepmind/funsearch` — six directories. Five hold results:

- `cap_set/` (the priority functions and the 512-point cap set as a text file), `admissible_set/`, `bin_packing/` (with an evaluation suite reproducing the paper's table), `cyclic_graphs/`, `corner_free_sets/`.

The sixth, `implementation/`, is the method itself: about 900 lines of single-threaded Python.

- `funsearch.py` — the pipeline; the problem is one Python file in which the decorator `@funsearch.evolve` marks the function to evolve and `@funsearch.run` marks the scoring entry point (a decorator is a label attached to a function definition).
- `programs_database.py` — islands, score-signature clusters, softmax sampling, periodic reset of the worst half.
- `config.py` — the paper's defaults: 2 programs per prompt, 10 islands, reset every 4 hours, 15 samplers, 140 evaluators.

Two deliberate stubs: LLM and Sandbox

Two classes in the released code are stubs — placeholder methods that raise an error until the user supplies an implementation:

- `sampler.LLM`: `raise NotImplementedError('Must provide a language model.')`
- `evaluator.Sandbox`: `raise NotImplementedError('Must provide a sandbox for executing untrusted code.')` — a sandbox is an isolated execution environment, so machine-written code cannot damage the host.

The README on scope: the directory “does not contain language models for generating new programs, the sandbox for executing untrusted code, nor the infrastructure for running FunSearch on our distributed system,” and is “intended ... for adapting it for use with any available language models, sandboxes, and distributed systems.”

Reproducible in design, not runnable out of the box: the user supplies an LLM binding, a sandbox, and parallelism.

**AlphaEvolve: from one function to
whole code files**

Evolve blocks, two Gemini models, diffs

Novikov et al., DeepMind, 14 May 2025 (arXiv:2506.13131).

- The user marks evolvable regions of a real code file:

```
# EVOLVE-BLOCK-START
... code the system may rewrite ...
# EVOLVE-BLOCK-END
```

Everything outside is fixed scaffolding. Several regions can evolve together — one discovered training-pipeline improvement required 15 coordinated mutations across optimizer, loss, and hyperparameters.

- Two Gemini models generate: Flash (fast, cheap — breadth) and Pro (slower — occasional higher-quality suggestions).
- Mutations are emitted as SEARCH/REPLACE diffs (not full rewrites):

```
<<<<<<< SEARCH
... old code chunk ...
=====
... new code chunk ...
>>>>>>> REPLACE
```

The host applies the patch by exact-string substitution on the file.

How the search learns: the prompt

The model's weights are frozen — the only thing that improves from step to step is *what the model is shown*. So each prompt is assembled to carry the current state of the search:

- the best programs found so far, with their scores — the model sees what already works;
- the problem description, relevant equations, literature, human hints;
- the actual output of running the previous candidate — its error messages, printed numbers, timings — so the model fixes a concrete failure instead of guessing.

The system can even evolve the wording of its own prompt instructions (*meta-prompting*). All “learning” lives here, in the text handed to a fixed model.

Filtering and storing candidates: cascade and database

Evaluation cascade — *problem*: fully scoring a candidate can take minutes to hours, and most candidates are bad.

- Score each candidate on a cheap test first; only survivors advance to the expensive tests. Bad candidates die after the cheap stage.
- Several metrics can be scored at once — optimizing several together often improves even the single target metric.

Program database — *problem*: keeping only the single best program collapses the search onto one approach and sticks in a local optimum. Keep a *diverse* set instead, two ways combined:

- **MAP-Elites**: bin programs by a few cheap features (length, a diversity measure); keep the best in each bin — many distinct styles survive.
- **Islands**: several separate populations evolve in parallel, exchanging members only occasionally, so independent lines of attack do not homogenize.

AlphaEvolve vs. FunSearch (the paper's comparison)

The same loop as FunSearch, with every component upgraded two years later — the paper's own comparison:

FunSearch (2023)	AlphaEvolve (2025)
evolves a single function	evolves an entire code file
up to 10–20 lines, Python only	up to hundreds of lines, any language
fast evaluation needed (≤ 20 min, 1 CPU)	evaluation may take hours, on accelerators
millions of LLM samples	thousands of samples suffice
small code model; no gain from bigger	benefits from frontier LLMs
minimal prompt context	rich context: problem, literature, feedback
one score	several scores optimized simultaneously

More compute per candidate, far fewer candidates.

4×4 over $\mathbb{C}$ in 48 multiplications

- Strassen recursion: rank 49 for $\mathcal{T}_4$, over any field. AlphaTensor's 47 needs $1 + 1 = 0$ (characteristic 2). Over characteristic 0, rank below 49 was open for 56 years.
- AlphaEvolve found rank **48** over $\mathbb{C}$.
 - What evolves: a gradient-based search program (loss, optimizer, initialization). Run it $\rightarrow$ numerical decomposition $\rightarrow$ round entries to integers or half-integers $\rightarrow$ verify the rounded identity exactly.
 - Applies recursively. Coefficients are complex, but the verified identity is an algebraic equality, so it multiplies real matrices too.
- Same agent — no tensor-specific game. Only the evaluator and the prompt change between problems.

What the rank-48 scheme concretely is

Three 16×48 complex matrices U, V, W : the 16 rows index the entries of A , of B , of the output C (each 4×4 , flattened); the 48 columns are the 48 products.

- For $r = 1, \dots, 48$: $m_r = (U_{:,r} \cdot a) (V_{:,r} \cdot b)$ — a complex combination of A 's entries times one of B 's. Then $c_{ik} = (W\text{-row}) \cdot m$.
- Every coefficient is one of **nine values**: $\{0, \pm\frac{1}{2}, \pm\frac{i}{2}, \pm\frac{1}{2} \pm \frac{i}{2}\}$ — this is what “half-integer complex” means.

One of the 48 products (common $\frac{1}{2}$ factored out):

$$m_1 = \frac{1}{2} \left[(1+i)(a_{11}+a_{12}) + (1-i)(a_{21}+a_{22}+a_{31}+a_{32}+a_{41}+a_{42}) \right] \\ \cdot \frac{1}{2} \left[-b_{11} - b_{21} + b_{31} - i b_{41} \right].$$

Full scheme: $3 \times 16 \times 48 = 2304$ numbers; one output sums 28 of the 48 products. The certificate is the point — expand the 48 products and the 4×4 identity checks out ([DeepMind's notebook](#), re-run for this lecture).

14 formats improved; the 4×4 record over time

- Improved the known rank for 14 matrix-multiplication formats — e.g. $\langle 3, 4, 7 \rangle$: $66 \rightarrow 63$; $\langle 4, 5, 6 \rangle$: $93 \rightarrow 90$.
- Matched or surpassed the best known rank for all formats with $m, n, p \leq 5$.

The 4×4 record over time:

1969	Strassen, human	49	any field
2022	AlphaTensor, reinforcement learning	47	$\mathbb{Z}_2$ only
2025	AlphaEvolve, coding agent	48	over $\mathbb{C}$ (char. 0)
2025	humans, from AlphaEvolve's scheme	48	rational (arXiv:2506.13242)

The last row: Dumas–Pernet–Sedoglavic applied an *isotropy* of the tensor (a symmetry — a simultaneous change of basis on the A, B, C slots taking one valid scheme to another) chosen to land on rational coefficients, so 48 now works over $\mathbb{Q}, \mathbb{R}$, not only $\mathbb{C}$.

Over 50 open problems: 75% matched, 20% improved

- Over 50 open problems in analysis, combinatorics, number theory, and geometry, formulated with Javier Gómez-Serrano and Terence Tao.
- For each problem, the agent evolves a *search heuristic program*: given a time budget and the best known construction, output a better one.
- Outcome: $\sim 75\%$ rediscovered the best known construction; $\sim 20\%$ improved on it.
- Follow-up at 67 problems: Georgiev, Gómez-Serrano, Tao, Wagner, [arXiv:2511.02864](https://arxiv.org/abs/2511.02864) (November 2025).

Three of the improvements

- **Kissing number, dimension 11.** The kissing number is how many non-overlapping unit spheres can simultaneously touch one central unit sphere. AlphaEvolve found a configuration of **593** spheres; the previous record was 592.
- **Erdős's minimum overlap.** Split $\{1, \dots, 2n\} = A \sqcup B$ in half. Overlap of the split: $\max_k |\{(a, b) \in A \times B : a - b = k\}|$. Let $M(n) = \min$ overlap over splits; the asymptotic constant $c = \lim_n M(n)/n$ is open. Upper bound: $0.380926 \rightarrow 0.380924$.
- **Packing.** Circles in the unit square ($n = 26$), max sum of radii: $2.6340 \rightarrow$ **2.6358** (this is the source of the 2.635 record). Unit regular hexagons in a smallest enclosing regular hexagon, outer side: $4.000 \rightarrow$ **3.942**. Further records on circle-packing variants, Heilbronn-type problems, and autocorrelation inequalities.

Each problem was configured only through its evaluator and prompt — no problem-specific system was built for any of them.

Datacenter scheduling: 0.7% of fleet compute

- Target: a scoring heuristic inside Borg (Google's cluster scheduler) that re-ranks candidate machines for each arriving job.
- The evolved function is one line — a sum of four terms built from each machine's leftover processor and memory capacity.
- DeepMind chose it over a deep-RL alternative for its “interpretability, debuggability, predictability and ease of deployment” (the announcement).
- In fleet-wide production: recovers on average 0.7% of Google's compute resources that would otherwise be stranded.

Gemini's training kernels; FlashAttention; a TPU circuit

- **Gemini training kernels.** A tiling heuristic — how each multiplication is cut into hardware-sized blocks — for the matmul kernels used to train Gemini. 23% average kernel speedup; $\sim 1\%$ off total Gemini training time; tuning turnaround cut from months to days.
- **FlashAttention** (the standard fast attention implementation in transformers): 32% kernel speedup, obtained by editing the compiler's intermediate representation.
- **TPU circuit.** A simplification of Verilog code (the hardware-description language) removed unnecessary bits in an arithmetic unit inside the TPU's matmul circuit. Validated by the designers, slated for an upcoming TPU. Caveat (from the paper): downstream synthesis tools caught it independently.

Each target had an exact automated evaluator: correctness checks plus measured speed or resource metrics.

AlphaEvolve's stated limitation

From the paper:

"The main limitation of AlphaEvolve is that it handles problems for which it is possible to devise an automated evaluator."

- Tasks requiring manual experimentation — much of the natural sciences — are out of scope.
- Proposed next step (not done in the paper): use the programs and results AlphaEvolve discovers as training data for the next base model, so the model itself absorbs the capability.



Alexander Novikov — first author of the AlphaEvolve white paper; research scientist, Google DeepMind. Photo: X profile.

AlphaEvolve: results only

`github.com/google-deepmind/alphaevolve_results` contains a notebook with the mathematical discoveries of the white paper's Section 3 and code verifying their correctness — and states:

“This repository does not contain the code to run AlphaEvolve.”

- No agent code, no prompts, no program database, no pipeline.
- Access to the system goes through Google. The announcement blog: “We’re planning an Early Access Program for selected academic users” and “exploring possibilities to make AlphaEvolve more broadly available,” with an interest-registration form.

Per system: AlphaTensor ships its outputs; FunSearch ships outputs and the method as readable reference code; AlphaEvolve ships outputs only.

OpenEvolve: an open re-implementation of AlphaEvolve

OpenEvolve: an open re-implementation of the AlphaEvolve design

- The openevolve repository under the `algorithmicsuperintelligence` GitHub org; created and maintained by Asankhaya Sharma. Apache 2.0.
- `pip install openevolve`; version 0.2.27; ~6,500 GitHub stars (June 2026).
- Model-agnostic: talks to any OpenAI-compatible API endpoint — OpenAI, Gemini, or self-hosted open models served by vLLM or Ollama.
- Command-line tool or Python library; about 9,600 lines of Python.



Asankhaya Sharma — creator and maintainer of OpenEvolve. Photo: GitHub profile.

OpenEvolve from the source: evolve blocks, diffs, ensemble

- **Evolve blocks.** The user marks evolvable regions with the same `# EVOLVE-BLOCK-START / # EVOLVE-BLOCK-END` comments AlphaEvolve uses.
- **Diffs.** Default mutation mode is diff-based; the parsing pattern in `config.py` is AlphaEvolve's `<<<<<< SEARCH / ===== / >>>>>> REPLACE` format verbatim. Default is diff mode (`diff_based_evolution: true`); switching to full-rewrite mode is a manual config choice.
- **Model ensemble.** A weighted list of models; each generation call picks one by weighted random choice — the Flash/Pro split generalized to arbitrary ensembles. Selection is seeded, so runs are repeatable.
- **Prompts** carry the current program, 3 top programs and 2 diverse programs from the database, optional evolved meta-instructions, and up to 20 KB of *artifacts* — execution output (errors, timings, printouts) from the previous evaluation fed back to the model.

From the source: an evolve block, a diff

Initial program with one evolvable region:

```
# EVOLVE-BLOCK-START
def priority(v, n):
    return 0.0
# EVOLVE-BLOCK-END

def solve(n): ...                # fixed scaffolding
```

One LLM response (AlphaEvolve SEARCH/REPLACE format, verbatim):

```
<<<<<<< SEARCH
def priority(v, n):
    return 0.0
=====
def priority(v, n):
    return sum(c * (i + 1) for i, c in enumerate(v)) % n
>>>>>>> REPLACE
```

- `openevolve/utils/code_utils.py` (`extract_diffs`, `apply_diff`) extracts SEARCH/REPLACE blocks and applies them by exact-string substitution. If SEARCH is not found verbatim the candidate is discarded as a parse failure.

Where mutation happens: one iteration in code

One iteration = `run_iteration_with_shared_db` in `openevolve/iteration.py`:

```
parent, inspirations = database.sample(...)
prompt    = prompt_sampler.build_prompt(...)    # code+scores -> text
response  = await llm_ensemble.generate_with_context(...)
diffs     = extract_diffs(response)             # SEARCH/REPLACE
child     = apply_diff(parent.code, ...)        # patched parent
metrics   = await evaluator.evaluate_program(child, ...)
database.add(child)                            # MAP-Elites insert
```

- The mutation operator is the one LLM call. There is no hand-coded genetic operator in the package: no `mutate()`, no `crossover()` — no bit flips, no syntax-tree edits.
- The child = the parent's code with the diff applied; `parent_id` is recorded — a run leaves an evolution tree.
- Recombination happens inside the model, if at all: the prompt shows top and inspiration programs, and the model may merge their ideas into its diff. No function combines two programs.

What the model sees each iteration

System message, complete (BASE_SYSTEM_TEMPLATE): "You are an expert software developer tasked with iteratively improving a codebase. Your job is to analyze the current program and suggest improvements based on feedback from previous attempts. Focus on making targeted changes that will increase the program's performance metrics."

User message: a fixed skeleton of slots (DIFF_USER_TEMPLATE):

```
# Current Program Information
- Current performance metrics: {metrics}
- Areas identified for improvement: {improvement_areas}
{artifacts}
# Program Evolution History
{evolution_history}
# Current Program
{current_program}
# Task
...You MUST use the exact SEARCH/REPLACE diff format...
IMPORTANT: Do not rewrite the entire program - focus on
targeted improvements.
```

How the prompt slots are filled

- `{improvement_areas}` is computed, not static (`_identify_improvement_areas`): the parent's fitness — computed excluding the feature dimensions — is compared with the previous attempt's, and a sentence is emitted: fitness improved / declined / stable; plus the program's current region of the feature grid.
- `{artifacts}`: the previous evaluation's execution output — error messages, timings, printouts — capped at 20 KB. The model reads its candidate's own runtime feedback.
- `{evolution_history}`: previous attempts with their scores, the top programs, and the inspiration programs (3 top + 2 diverse by default, all from the parent's island).
- `{current_program}`: the parent's full code.

Selection pressure enters the prompt as text: scores, score changes, and runtime errors are written into the next generation's instructions.

From the source: an evaluator

Contract. `evaluate(path)` returns a dict of metric $\rightarrow$ value; `combined_score` is what selection optimizes.

```
def evaluate(path):
    spec = importlib.util.spec_from_file_location("cand", path)
    cand = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(cand)
    try:
        signal.alarm(300)
        S = cand.solve(n=8)
        signal.alarm(0)
    except Exception as e:
        return {"combined_score": 0.0, "valid": 0.0, "error": str(e)}
    if not is_cap_set(S):
        return {"combined_score": 0.0, "valid": 0.0}
    return {"combined_score": float(len(S)), "valid": 1.0,
            "complexity": len(open(path).read())}
```

- Cascade: define `evaluate_stage1/2/3` for early culling; every stage must return every metric key (placeholder `0.0`) — a missing key silently drops the candidate at database insert.

MAP-Elites: one champion per cell of a feature grid

Mouret–Clune 2015, *Illuminating search spaces by mapping elites*. Each program carries two kinds of numbers:

- a **fitness** — the evaluator's score, the thing being maximized;
- **feature coordinates** — descriptive axes, here: code length (“complexity”) and edit-distance diversity.

Cut the feature space into a grid. Each cell keeps **one** program: the best-scoring one that landed there.

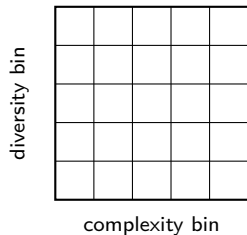
- Insertion: empty cell → occupy; occupied → replace only if strictly better.
- The feature axes are coordinates, not objectives: each cell still maximizes the single fitness.
- Best-of-population selection collapses onto one kind of program; the grid keeps the best program of each kind alive as a future parent.

The database: MAP-Elites grid $\times$ 5 islands

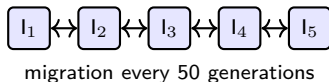
Each island carries a MAP-Elites grid; the database keeps 5 islands.

cell = best program in that bin

one island



5 islands



- Defaults: 10 bins per axis, complexity $\times$ edit-distance diversity; population 1000 per island, archive 100. Any evaluator metric can replace a feature axis.
- **Insert.** Compute the candidate's feature bin; replace the cell's occupant only if the candidate scores higher.

The grid in code: one store, id-only views

- The grid is not an array. Each island holds a dictionary {cell key $\rightarrow$ program id}; the key joins the bin indices: coordinates $[3, 7] \rightarrow$ "3-7". Unoccupied cells are simply absent.
- Placing a program: each feature value (an evaluator metric, or `len(program.code)` for complexity) is min-max scaled by the running min/max observed so far, multiplied by the bin count, and clamped to a bin index.
- One central store programs: {id $\rightarrow$ Program} holds the objects; grids, island sets, and the archive hold only id strings.
- Fitness never mixes with features: the comparison score explicitly excludes the metrics used as feature axes (source comment: "excludes MAP-Elites feature dimensions").
- Three kinds of "best", kept separately: the per-cell champion; the **archive**, a global top-100 elite pool; and the absolute best program, tracked on its own — docstring: "to ensure it's never lost."

Choosing the parent: three tiers

Each iteration draws one random number $r \in [0, 1)$ and picks the parent accordingly (defaults from `config.py`):

- $r < 0.2$: a **random member** of the current island — exploration;
- $0.2 \leq r < 0.9$: an **elite from the archive** — exploitation;
- $r \geq 0.9$: a **fitness-weighted** island member.

The inspiration programs — the additional scored programs shown to the model in the prompt — are drawn only from the parent's own island.

Islands: independent grids, coupled only by migration

- Each island evolves independently and carries its **own** MAP-Elites grid (`island_feature_maps`: one dictionary per island).
- Prompts never show programs from other islands; migration is the only coupling between islands.
- Migration: at each migration event, the island's top fraction is **copied** to its two ring neighbours, islands $(i + 1) \bmod N$ and $(i - 1) \bmod N$.
- A copied program is marked `migrant` and is never migrated again. The guard records a real failure — source comment: “Program `cb5d07f2` had 183 descendant copies by iteration 850” — identical copies all map to the same cell, so only one survives and the rest are wasted evaluations.

Evaluation cascade, operations, novelty judge

Evaluation cascade — *problem*: fully scoring a candidate may take minutes, and most candidates are bad.

- The user's evaluator may define stages `evaluate_stage1/2/3`.
- OpenEvolve detects them at startup and runs them in order; candidates failing an early stage skip the rest.
- Per-candidate timeout: 300 s.

Operations — single-machine pipeline; defaults sized for a workstation, not a datacenter.

- Async controller; process-based parallel workers.
- Checkpoint every 100 iterations; resume from any checkpoint.
- Deterministic seeding (default 42) — runs are repeatable.
- Browser-based visualizer of the program tree.

Beyond the AlphaEvolve paper. An optional LLM novelty judge (adapted from Sakana AI's ShinkaEvolve) rejects candidates whose behavior is not meaningfully different from existing programs.

OpenEvolve on circle packing, $n = 26$

Documented run: sum of radii **2.634292** (99.97% of AlphaEvolve's 2.635, matching but not beating). The discovered program:

```
def construct_packing():
    n, centers, count = 26, np.zeros((n, 2)), 0
    # (1) Layout: graded radii on a 5x5 grid, alternating column offset
    radii = np.linspace(0.12, 0.05, n)
    xs = ys = np.linspace(0.15, 0.85, 5)
    for i in range(5):
        for j in range(5):
            centers[count] = [xs[i] + 0.05*(j%2), ys[j]]; count += 1

    # (2) Refine with SLSQP: maximize sum of radii subject to
    #     non-overlap + boundary constraints
    x0 = np.concatenate([centers.flatten(), radii])
    result = minimize(objective, x0, method="SLSQP",
                      bounds=[(0,1)]*(2*n) + [(0.03, 0.2)]*n,
                      constraints={"type":"ineq", "fun": constraint})
    return result.x[:2*n].reshape(n,2), result.x[2*n:]
```

- The LLM contributes the *architecture* (graded-radii grid + SLSQP refinement) and good initial guesses; the optimal coordinates come from `scipy.optimize`.

Running OpenEvolve

A minimal run needs three files — an initial program with one evolve block, an evaluator returning a score dictionary, a config naming the model — and a served model:

```
pip install openevolve
python openevolve-run.py initial_program.py evaluator.py \
    --config config.yaml --iterations 50
```

- With a self-hosted open-weight model behind vLLM, the loop runs with no external API.
- Process isolation and timeouts are built in; a hardened security sandbox is not.
- The requirement of a cheap, exact evaluator is inherited, not removed.

Run: deciding Littlewood–Richardson positivity — the data

For partitions λ, μ, ν (weakly decreasing lists of positive integers), $c_{\mu\nu}^{\lambda}$ is a nonnegative integer, computable exactly by a counting rule — the Littlewood–Richardson (LR) rule. (It is the coefficient of s_{λ} in $s_{\mu}s_{\nu}$ for Schur polynomials: background.)

One data point = one triple with its exactly computed label:

λ	μ	ν	$c_{\mu\nu}^{\lambda}$	label
(3, 2, 1)	(2, 1)	(2, 1)	2	1
(3, 3)	(2)	(2, 2)	0	0

The second triple passes every cheap sanity check (sizes add up, μ and ν fit inside λ , ...) and is still zero — a **hard zero**. Hard zeros are what the run is after.

Dataset: 700 training triples ($|\lambda| \leq 12$); 300 test triples, deliberately larger ($|\lambda|$ 13–16).
Score of a predicate: accuracy on positives and on zeros, averaged, on the test triples.

The seed program; what one iteration does

The evolve block is one Python predicate. The seed: four cheap necessary conditions (abridged from the run's actual file):

```
def is_positive(lam, mu, nu):          # EVOLVE-BLOCK
    # contains(a, b): b[i] <= a[i] for every row i
    if sum(lam) != sum(mu) + sum(nu):    return False
    if not (contains(lam,mu) and contains(lam,nu)): return False
    if lam[0] > mu[0] + nu[0]:            return False
    if len(lam) > len(mu) + len(nu):      return False
    return True
```

- Seed score: every positive right, every hard zero wrong — test score 0.667.
- One iteration: OpenEvolve puts the best programs so far, with their scores, into a prompt; Qwen-7B rewrites the block; the evaluator runs the new predicate on the 700 training triples and stores its score in the program database.
- 5,000 iterations.

The champion program after 5,000 iterations

Keeps the seed checks; adds two loops (abridged, as evolved):

```
for i in range(len(lam)):                # Weyl inequalities
    for j in range(i + 1):
        if j < len(mu) and i-j < len(nu):
            if lam[i] > mu[j] + nu[i-j]: return False
for k in range(len(lam)):                # dominance partial sums
    if sum(lam[:k]) > sum(mu[:k]) + sum(nu[:k]): return False
```

- These are the Weyl inequalities $\lambda_{i+j+1} \leq \mu_{i+1} + \nu_{j+1}$ and the dominance condition — the two simplest layers of the **Horn system** of inequalities characterizing positivity (Klyachko, Knutson–Tao).
- Test score $0.667 \rightarrow 0.814$; no true positive is ever rejected — every added condition is genuinely necessary. The deeper Horn layers were not found.
- Also evolved: blocks that check nothing — e.g. a “conjugate” step that re-sorts an already sorted tuple.
- An earlier run over-weighted hard zeros: a reject-most-positives predicate won. Rebalanced before this run.

Run: Naruse's skew hook-length formula — the target

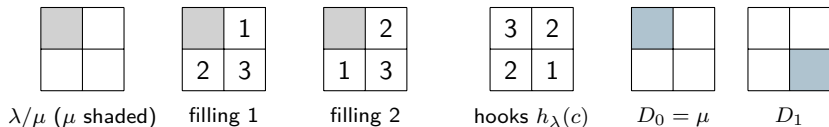
The **Young diagram** of λ has λ_i cells in row i ; the skew shape λ/μ removes μ 's cells from the top-left. $f^{\lambda/\mu}$ counts the **standard Young tableaux** of the shape: fillings with $1, \dots, |\lambda/\mu|$, increasing along rows and down columns.

- Straight shapes: $f^\lambda = |\lambda|! / \prod_c h(c)$ (Frame–Robinson–Thrall 1954); the **hook length** $h(c)$ counts the cells to the right of c , plus the cells below c , plus 1.
- Skew shapes have no such product formula. Naruse (2014):

$$f^{\lambda/\mu} = |\lambda/\mu|! \sum_D \prod_{c \in [\lambda] \setminus D} \frac{1}{h_\lambda(c)},$$

summed over **excited diagrams** D : copies of μ 's cell set pushed by repeated diagonal moves $(i, j) \rightarrow (i+1, j+1)$ inside λ .

The formula on one example: $\lambda = (2, 2)$, $\mu = (1)$



- The skew shape has 3 cells; exactly two standard fillings, so $f^{\lambda/\mu} = 2$.
- Excited diagrams of μ : the unmoved $D_0 = \{(1, 1)\}$ and $D_1 = \{(2, 2)\}$, its one diagonal move. The product skips D 's cells:

$$f^{\lambda/\mu} = 3! \left(\frac{1}{2 \cdot 2 \cdot 1} + \frac{1}{3 \cdot 2 \cdot 2} \right) = 6 \left(\frac{1}{4} + \frac{1}{12} \right) = 2.$$

What evolves: the family generator

The shell is frozen in the evaluator: for each shape it computes, in exact arithmetic,

$$F = |\lambda/\mu|! \sum_{D \in \text{family}(\lambda, \mu)} \prod_{c \in [\lambda] \setminus D} \frac{1}{h_\lambda(c)},$$

and checks $F = f^{\lambda/\mu}$ (true value computed independently). Score: fraction of shapes matched exactly — 700 training, 300 larger test shapes. Only the family generator evolves; the seed returns μ unmoved:

```
def candidate_family(lam, mu):          # EVOLVE-BLOCK
    return [frozenset((i, j) for i in range(len(mu))
                      for j in range(mu[i]))]
```

- Seed: exact only when no diagonal move exists — test exact rate 0.15.
- The prompt never names excited diagrams. Probe before evolving: asked point-blank for a skew hook-length formula, the model gives a wrong one (a product formula attributed to “RSK”, wrong hooks).

Output: three runs, the family is not found

- Diff mode, 600 iterations: 600/600 “No valid diffs found” (7B model); all later runs use full-rewrite mode.
- Full rewrite, high temperature, 5,000 iterations: best = seed; the children collapsed onto one brute-force subset-search program scoring below the seed.
- Full rewrite, low temperature, 5,000 iterations: all $\sim 5,000$ children scored exactly the seed's 0.3128 — verbatim copies of the seed.

The excited-diagram family was neither recalled (the probe) nor discovered (the runs).

Run: circle packing $n = 26$ — task, inputs, output

Task: place 26 disjoint circles in the unit square, maximizing the sum of radii (the problem that opened this lecture).

- **Evaluator:** the exact check — every circle inside the square, every pair disjoint — returning the sum of radii; made deterministic by seeding the RNG.
- **Seed program:** the 5×5 grid of radius-0.1 circles plus one in a gap, sum 2.541.
- **What evolves:** the whole constructor `construct_packing()` $\rightarrow$ $26 \times (x, y, r)$ (full rewrite each step).

Output: best deterministic sum **2.541** — equal to the grid seed. The 7B model produced no valid packing beating the grid, and about **97%** of its candidates were invalid (overlaps or out-of-bounds). Reaching AlphaEvolve's 2.6358 needs the graded-radius layout, which a 7B on one GPU did not find: the loop runs and scores correctly on open weights, but a small model does not reach the frontier record on this geometric problem.

The evaluator must be exact *and* deterministic

Caught while running OpenEvolve on the 26-circle problem (XMU HPC, June 2026). The evolved program placed the circles, then ran a local optimizer using an *unseeded* random number generator.

- The same program scored a radius sum of 2.56 on one evaluation and 2.47 on the next.
- A noisy score breaks selection: the search cannot tell a real improvement from luck, and it stalls.
- Fix: seed the random generator inside the evaluator, so every candidate is scored reproducibly.

The “cheap exact evaluator” precondition silently includes *deterministic*: a score that wobbles is not an exact score — the most practical lesson from running the method by hand.

Run: a representation-theory selector — task, inputs, output

Task: resolve the last unknown of a correspondence from our own research (painted bipartition $\rightarrow$ local system, type D) — at each column a 2-bit choice (sign, swap) picking which of four candidate values to emit.

- **Evaluator:** assemble the full local system from the candidate's per-column choices, exact-match against 28,384 oracle-verified pairs; headline metric the exact-match rate on held-out *larger* instances.
- **Seed program:** the trivial selector (always the canonical choice).
- **What evolves:** one function `select(rec)  $\rightarrow$  {column: (sign, swap)}`.

Output: best held-out exact match about **23%** (orbit ceiling 90%). The search **independently recovered the predicted program shape** — a state carried left to right across the columns (parities of cumulative symbol counts, an alternating counter) — but found no clean rule. A second independent method (after an earlier search over a hand-built grammar) reaching the same wall corroborates that the correspondence is irreducibly the recursion.

**PatternBoost: local search plus a
small transformer**

PatternBoost: local search plus a small transformer

Charton (Meta), Ellenberg (Wisconsin), Wagner (Worcester Polytechnic), Williamson (Sydney), *PatternBoost: constructions in mathematics with a little help from AI*, [arXiv:2411.00566](https://arxiv.org/abs/2411.00566) (November 2024).

Input a score function; an encoding of constructions as token sequences; a classical **local search** for the problem (start from a construction, repeatedly make small changes that raise the score, from many random starts).

Output record or near-record constructions.

Objective maximize the score, alternating two phases.



Georgie Williamson —
representation theorist, University
of Sydney; PatternBoost co-author.

Photo: course asset.

The worked example: triangle-free graphs on 20 vertices

Maximize the number of edges of a 20-vertex graph containing no triangle. The answer is classical (Mantel 1907): $\lfloor 20^2/4 \rfloor = 100$, achieved by the complete bipartite graph $K_{10,10}$ — the method is demonstrated on known ground.

- **Encoding:** the upper-triangular part of the adjacency matrix, flattened row by row, rows separated by commas — a 3×3 matrix with rows 010, 01, 0 becomes the token string 010,01,0. For $n = 20$: 190 bits.
- **Local phase:** start from the empty graph; delete an edge from some triangle until none remains; then greedily add random edges that create no triangle.
- 40,000 local-search runs: best score 99 — “achieved quite rarely, by two lucky runs out of the 40,000 tries”; 100 is never reached.

One transformer round on the worked example

- Train a small transformer on the top 25% of the 40,000 scored graphs, encoded as token strings.
- Sample 37,000 new graphs from the trained model; run the same local search from each of them.
- Result: score 99 appears 47 times (twice in the previous round); the optimum 100 appears **46** times.
- After 5 more loops, “the model quickly learns to produce essentially only complete bipartite graphs” — the Mantel extremal structure.

The two phases

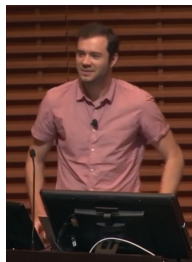
Stated aim of the paper: “a useful and simple tool for working mathematicians, one which does not require extensive expertise in machine learning or access to industry-level computing power.”

1. **Local phase.** Run classical local search (greedy plus repair moves) from many random seeds; collect a large pool of scored constructions.
 2. **Global phase.** Train a small transformer on the best fraction of the pool (top 25% in the worked example), encoded as token sequences — e.g. a graph's upper-triangular adjacency matrix flattened row by row. Sample new sequences; use them as *seeds* for the next local phase. Repeat 10–50 times.
- The transformer never sees the score function; it learns the distribution of good constructions and proposes more from it.
 - No reinforcement learning, no frontier model, no million-program budget.

Makemore: a minimal GPT-2-style transformer

Makemore — Andrej Karpathy's minimal decoder-only transformer, the block structure of GPT-2 (OpenAI's 2019 language model) at small size:

- each token gets a learned embedding vector plus a learned position embedding;
- 2–6 identical blocks; each = **masked self-attention** (every token attends only to *earlier* tokens; 4 heads, dimension 16–128) then a two-layer fully connected network, each with a residual connection (the input added back to the output);
- a final linear layer: at each position, a probability distribution over the vocabulary;
- training = next-token prediction on the best constructions; generation = sample token by token, decode into a candidate.



Andrej Karpathy — AI researcher; author of Makemore. Photo: Wikimedia Commons.

A 1992 hypercube conjecture refuted: 81 edges, not 82

Question (Erdős–Hamburger–Pippert–Weakley; Graham–Harary 1992): how many edges can be deleted from the d -dimensional hypercube without increasing its diameter (the largest graph distance between two vertices), which equals d ?

- The classical construction keeps $2^d + \binom{d}{\lfloor d/2 \rfloor} - 2$ edges; Niall Graham conjectured in 1992 (Graham–Harary, *Discrete Applied Mathematics* 37) that this is optimal.
- For $d = 6$ that count is 82. PatternBoost found a spanning subgraph of the 6-cube (a subgraph keeping all 64 vertices) with diameter 6 and **81** edges.
- The paper: “the first progress on this problem in 30 years.”



François Charton —
PatternBoost co-author;
research engineer at Meta AI
at the time; since 2025 at
Axiom Math. Photo: personal
homepage.

No-isosceles sets; Brualdi–Cao permanents

The paper tests nine problems. Two more outcomes:

- **No-isosceles point sets** — largest subset of an $n \times n$ grid with no three points forming an isosceles triangle (degenerate ones included):
 - $n = 64$: three months of problem-specific classical search had reached 108 points; one transformer loop on top reached **110** within days.
 - $n = 100$: 154 $\rightarrow$ **160**.
- **Brualdi–Cao permanents** — the permanent of a matrix sums, over all ways to pick one entry per row and per column, the product of those entries (the determinant without the $\pm$ signs). Maximize it over a structured class of $n \times n$ 0–1 matrices (those avoiding a fixed forbidden sub-pattern): here a handcrafted specialized algorithm still wins, 5,200,384 vs 5,101,230 at $n = 25$. The paper: “handcrafted specialized algorithms can still outperform generic ML methods.”

The six systems compared

The six systems: generator, what evolves, evaluator, record

	generator	what evolves	evaluator	record result
Wagner 2021	small net + cross-entropy	the object (bit string)	conjecture slack	19-vertex counterexample
AlphaTensor 2022	AlphaZero (MCTS + net)	the object (factor list)	residual = 0, count moves	4×4 mod 2 in 47
AlphaDev 2023	AlphaZero	the object (assembly)	exact tests + latency	sort3 in 17, in libc++
FunSearch 2023	frozen code LLM	a program (one function)	exact score of output	512-cap; $C \geq 2.2202$
PatternBoost 2024	small transformer	the object, via learned seeds	problem score	6-cube conjecture refuted
AlphaEvolve 2025	Gemini Flash + Pro	a code file (any language)	cascade, multi-metric	4×4 over $\mathbb{C}$ in 48

- Every system rests on exact, machine-checkable scoring.
- From 2021 to 2025 the generator changed, from move-level reinforcement learning to language models writing programs; FunSearch moved the search from object space to program space.

Records as certificates: the bound is proved, optimality stays open

- The 512-cap is a proof that $c_8 \geq 512$. The 81-edge subgraph disproves the 1992 conjecture. The rank-48 decomposition is a checkable identity of linear algebra.
- Verification is independent of the search that produced the object.
- Is $c_8 = 512$? Unknown. Is the rank of $\mathcal{T}_4$ over $\mathbb{Q}$ equal to 48? Unknown. Closing such gaps is a proof problem, not a construction problem.

Three machine records that humans extended by reading the output:

- the block symmetry extracted from FunSearch's program $\rightarrow$ the restricted search $\rightarrow C \geq 2.2202$;
- Kauers–Moosbauer's 95, grown out of AlphaTensor's 96;
- the rational 48, grown out of AlphaEvolve's complex 48.

Requirements: scoreable artifact, graded score, cheap evaluator

All six systems required:

- a finite, scoreable artifact;
- a graded score;
- an evaluator cheap enough to call thousands to millions of times.

Problems missing any of the three — theory building, definitions, proofs of optimality, anything needing laboratory experiments — are outside the scope these papers claim.

Primary sources: Wagner, AlphaTensor, AlphaDev

- A. Z. Wagner, *Constructions in combinatorics via neural networks*, [arXiv:2104.14516](#) (2021).
- A. Fawzi et al., *Discovering faster matrix multiplication algorithms with reinforcement learning*, Nature 610 (2022) 47–53.
- M. Kauers, J. Moosbauer, [arXiv:2210.04045](#) (2022); *Flip graphs for matrix multiplication*, ISSAC 2023.
- D. J. Mankowitz et al., *Faster sorting algorithms discovered using deep reinforcement learning*, Nature 618 (2023) 257–263.
- J. S. Ellenberg, D. Gijswijt, *On large subsets of $\mathbb{F}_q^n$ with no three-term arithmetic progression*, Annals of Mathematics 185 (2017) 339–343.

Primary sources: FunSearch, AlphaEvolve, PatternBoost

- B. Romera-Paredes et al., *Mathematical discoveries from program search with large language models*, Nature 625 (2024) 468–475.
- A. Novikov et al., *AlphaEvolve: a coding agent for scientific and algorithmic discovery*, Google DeepMind white paper, May 2025; [arXiv:2506.13131](#).
- B. Georgiev, J. Gómez-Serrano, T. Tao, A. Z. Wagner, *Mathematical exploration and discovery at scale*, [arXiv:2511.02864](#) (2025).
- F. Charton, J. Ellenberg, A. Z. Wagner, G. Williamson, *PatternBoost: constructions in mathematics with a little help from AI*, [arXiv:2411.00566](#) (2024).
- Repositories (read 2026-06-11): [github.com/google-deepmind/alphatensor](#), [google-deepmind/funsearch](#), [google-deepmind/alphaevolve_results](#), [algorithmicsuperintelligence/openevolve](#).